

1. Use create-react-app to set up a new project. Edit the App.js file to include a stateful component with useState. Add an input field and an element that displays text based on the input. Dynamically update the content as the user types.

Step-1:Create React Application

```
npx create-react-app my-dynamic-app
```

```
cd my-dynamic-app
```

step-2: modify app.js

```
import { useState } from 'react';

export default function App() {
  const [text, setText] = useState("");

  return (
    <div className="App">
      <h1>Dynamic Text Display</h1>
      <input
        value={text}
        onChange={e => setText(e.target.value)}
        placeholder="Type something..."
      />
      <p>You typed: {text}</p>
    </div>
  );
}
```

Dynamic Text Display

You typed: from hello

2. Develop a React application that demonstrates the use of props to pass data from parent component to child components. The application should include the parent component named App that serves as the central container for the application. Create two separate child components, Header: Displays the application title or heading. Footer: Displays additional information, such as copyright details or a tagline. Pass data (e.g., title, tagline, or copyright information) from the App component to the Header and Footer components using props. Ensure that the content displayed in the Header and Footer components is dynamically updated based on the data received from the parent component.

Step-1:Create React Application

```
npx create-react-app
```

Step 2:modify app.js

```

import { useState } from 'react';
import Header from './Header';
import Footer from './Footer';
import './App.css';

export default function App() {
  const [text, setText] = useState("");

  return (
    <div className="App">
      <Header title="Welcome to My React App" />

      <h1>Dynamic Text Display</h1>
      <input
        value={text}
        onChange={e => setText(e.target.value)}
        placeholder="Type Something..."
      />
      <p>You typed: {text}</p>

      <Footer
        tagline="Building great apps with React"
        copyright="@2026 MyApp, All Rights Reserved"
      />
    </div>
  );
}

```

Step 3:App.css

```

.App {
  text-align: center;
}

```

```
header, footer {  
  background: #282c34;  
  color: white;  
  padding: 10px;  
}
```

```
footer {  
  position: absolute;  
  bottom: 0;  
  width: 100%;  
}
```

Step 4: header.js

```
export default function Header({ title }) {  
  return (  
    <header>  
      <h1>{title}</h1>  
    </header>  
  );  
}
```

Step 5: footer.js

```
export default function Footer({ tagline, copyright }) {  
  return (  
    <footer>  
      <p>{tagline}</p>  
      <p>{copyright}</p>  
    </footer>  
  );  
}
```

Welcome to My React App

Dynamic Text Display

Hello Nandini

You typed: Hello Nandini

Building great apps with React

@2026 MyApp, All Rights Reserved

3.Create a Counter Application using React that demonstrates state management with the useState hook. Display the current value of the counter prominently on the screen. Add buttons to increase and decrease the counter value. Ensure the counter updates dynamically when the buttons are clicked. Use the useState hook to manage the counter's state within the component. Prevent the counter from going below a specified minimum value (e.g., 0). Add a "Reset" button to set the counter back to its initial value. Include functionality to specify a custom increment or decrement step value.

Step 1: Create a new React app

```
npx create-react-app counter-app
```

```
cd counter-app
```

step 2:modify App.js

```
import { useState } from 'react';
```

```
export default function App() {
```

```
  const [count, setCount] = useState(0);
```

```
  const [step, setStep] = useState(1);
```

```
  return (
```

```
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
```

```
      <h1>Counter Application</h1>
```

```
<h2>{count}</h2>
```

```
<button onClick={() => setCount(count + step)}>
```

```
  Increase by {step}
```

```
</button>
```

```
<button onClick={() => count - step >= 0 && setCount(count - step)}>
```

```
  Decrease by {step}
```

```
</button>
```

```
<button onClick={() => setCount(0)}>Reset</button>
```

```
<br /><br />
```

```
<input
```

```
  type="number"
```

```
  value={step}
```

```
  onChange={e => setStep(+e.target.value)}
```

```
  min="1"
```

```
/>
```

```
</div>
```

```
);
```

```
}
```

Counter Application

0

Increase by 1 Decrease by 1 Reset

1

4. Develop a To-Do List Application using React functional components that demonstrates the use of the `useState` hook for state management. Create a functional component named `ToDoFunction` to manage and display the to-do list. Maintain a list of tasks using state. Provide an input field for users to add new tasks. Dynamically render the list of tasks below the input field. Ensure each task is displayed in a user-friendly manner. Allow users to delete tasks from the list. Mark tasks as completed or pending, and visually differentiate them.

Step 1: Create a New React Application

```
npx create-react-app todo-app
```

```
cd todo-app
```

step 2: Modify App.js

```
import { useState } from 'react';

export default function App() {

  const [t, s] = useState([]), [v, sv] = useState("");

  return (

    <div style={{ textAlign: 'center' }}>

      <h3>To-Do</h3>

      <input value={v} onChange={e => sv(e.target.value)} />

      <button onClick={() => v && (s([...t, { i: Date.now(), v, d: 0 }]), sv(""))}>

        Add

      </button>

      {t.map(x => (

        <div key={x.i}>

          <span onClick={() => s(t.map(y => y.i === x.i ? { ...y, d: !y.d } : y))}

            style={{ textDecoration: x.d ? 'line-through' : '' }}>

            {x.v}

          </span>

          <button onClick={() => s(t.filter(y => y.i !== x.i))}> ✖ </button>

        </div>

      ))}

    </div>

  );

}
```

To-Do

Add

today is forum day 

today we had dosa 

5. Develop a React application that demonstrates component composition and the use of props to pass data. Create two components: **FigureList**: A parent component responsible for rendering multiple child components. **BasicFigure**: A child component designed to display an image and its associated caption. Use the **FigureList** component to dynamically render multiple **BasicFigure** components. Pass image URLs and captions as props from the **FigureList** component to each **BasicFigure** component. Style the **BasicFigure** components to display the image and caption in an aesthetically pleasing manner. Arrange the **BasicFigure** components within the **FigureList** in a grid or list format. Allow users to add or remove images dynamically. Add hover effects or animations to the images for an interactive experience.

Step 1: Create a New React Application

```
npx create-react-app figure-gallery
```

```
cd figure-gallery
```

step 2: Update App.js

```
import { useState } from 'react';
```

```
const Figure = ({ src, cap }) => (  
  <div style={{ textAlign: 'center' }}>  
    <img src={src} width="150" alt={cap} />  
    <p>{cap}</p>  
  </div>  
);
```

```
export default function App() {  
  const [f, setF] = useState([  
    { src: 'https://picsum.photos/200', cap: 'Image 1' },  
    { src: 'https://picsum.photos/200', cap: 'Image 2' }  
  ]);
```

```

return (
  <div style={{ textAlign: 'center' }}>
    <h2>Image Gallery</h2>

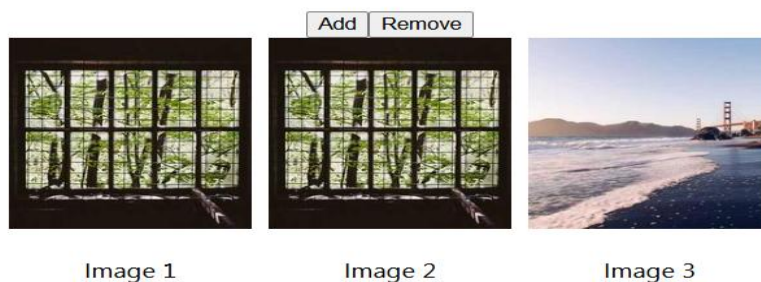
    <button onClick={() =>
      setF([...f, {
        src: `https://picsum.photos/200?${f.length}`,
        cap: `Image ${f.length + 1}`
      }])
    }>
      Add
    </button>

    <button onClick={() => setF(f.slice(0, -1))}>
      Remove
    </button>

    <div style={{ display: 'flex', justifyContent: 'center', gap: 10 }}>
      {f.map((x, i) => <Figure key={i} {...x} />)}
    </div>
  </div>
);
}

```

Image Gallery



6. Design and implement a React Form that collects user input for name, email, and password. Form Fields are Name, Email, and Password. Ensure all fields are filled before allowing form submission. Validate the email field to ensure it follows the correct email format (e.g., example@domain.com). Optionally enforce a minimum password length or complexity. Display error messages for invalid or missing inputs. Provide visual cues (e.g., red borders) to highlight invalid fields. Prevent form submission until all fields pass validation. Log or display the entered data upon successful submission (optional). Add a "Show Password" toggle for the password field. Implement client-side sanitization to ensure clean input.

Step 1: Create a New React Application

```
npx create-react-app react-form
```

```
cd react-form
```

Step 2: Update App.js

```
import { useState } from 'react';

export default function App() {
  const [f, setF] = useState({ n: "", e: "", p: "" });
  const [show, setShow] = useState(false);

  const valid =
    f.n &&
    /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(f.e) &&
    f.p.length >= 6;

  return (
    <div style={{ textAlign: 'center', marginTop: 40 }}>
      <h2>Form</h2>

      <input
        placeholder="Name"
        value={f.n}
        onChange={e => setF({ ...f, n: e.target.value })}
      /><br />

      <input
```

```

placeholder="Email"

value={f.e}

onChange={e => setF({ ...f, e: e.target.value })}

/><br />

<input

type={show ? 'text' : 'password'}

placeholder="Password"

value={f.p}

onChange={e => setF({ ...f, p: e.target.value })}

/><br />

<label>

<input type="checkbox" onChange={() => setShow(!show)} />

Show Password

</label><br /><br />

<button disabled={!valid} onClick={() => alert(JSON.stringify(f))}>

Submit

</button>

</div>

);

}

```

Form

☒ Show Password

7. Develop a React Application featuring a ProfileCard component to display a user's profile information, including their name, profile picture, and bio. The component should demonstrate flexibility by utilizing both external CSS and inline styling for its design. Display the following information: Profile picture, User's name, A short bio or description Use an external CSS file for overall structure and primary styles, such as layout, colors, and typography. Apply inline styles for dynamic or specific styling elements, such as background colors or alignment. Design the ProfileCard to be visually appealing and responsive. Ensure the profile picture is displayed as a circle, and the name and bio are appropriately styled. Add hover effects or animations to enhance interactivity. Allow the background color of the card to change dynamically based on a prop or state.

Step 1: Create a New React Application

```
npx create-react-app profile-card-app          cd profile-card-app
```

Step 2:Update App.js

```
import { useState } from 'react';

const Card = ({ name, bio, img }) => {
  const [bg, setBg] = useState('#eee');

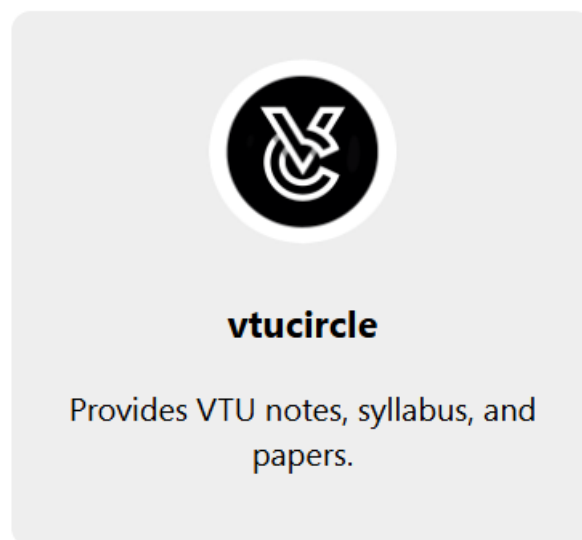
  return (
    <div
      onMouseEnter={() => setBg('#d1c4e9')}
      onMouseLeave={() => setBg('#eee')}
      style={{
        background: bg,
        padding: 20,
        width: 250,
        textAlign: 'center',
        borderRadius: 10,
        margin: '50px auto'
      }}
    >
      <img
        src={img}
        alt={name}
        style={{ width: 100, height: 100, borderRadius: '50%' }}
      />
    </div>
  )
}
```

```

    />
    <h3>{name}</h3>
    <p>{bio}</p>
  </div>
);
};

export default function App() {
  return (
    <Card
      name="vtucircle"
      bio="Provides VTU notes, syllabus, and papers."
      img="https://vtucircle.com/wp-content/uploads/2024/11/cropped-vtucircle_icon-1.png"
    />
  );
}

```



8. Develop a Reminder Application that allows users to efficiently manage their tasks. The application should include the following functionalities: Provide a form where users can add tasks along with due dates. The form includes task name, Due date, An optional description. Display a list of tasks dynamically as they are added. Show relevant details like task name, due date, and completion status. Include a filter option to allow users to view all Tasks and Display all tasks regardless of status. Show only tasks marked as completed. Show only tasks that are not yet completed

Step 1: Create a New React Application

```
npx create-react-app react-reminder-app          cd react-reminder-app          npm start
```

Step 2: Update App.js

```
import { useState } from 'react';

export default function App() {
  const [t, s] = useState([]), [n, sn] = useState(""), [d, sd] = useState(""), [f, sf] = useState('all');

  return (
    <div style={{ textAlign: 'center' }}>
      <h3>Reminder</h3>

      <input value={n} onChange={e => sn(e.target.value)} placeholder="Task" />
      <input type="date" value={d} onChange={e => sd(e.target.value)} />
      <button onClick={() => n && d && (s([...t, { i: Date.now(), n, d, c: 0 }]), sn(""), sd(""))}>
        Add
      </button>

      <br />

      {[ 'all', 'done', 'pending' ].map(x =>
        <button key={x} onClick={() => sf(x)}>{x}</button>
      )}

      {t.filter(x => f === 'all' ? 1 : f === 'done' ? x.c : !x.c)
        .map(x => (
          <div key={x.i}>
```

```

    {x.n} ({x.d}) {x.c ? '✓' : ''}

    <button onClick={() => s(t.map(y => y.i === x.i ? { ...y, c: !y.c } : y))}>✓</button>

    <button onClick={() => s(t.filter(y => y.i !== x.i))}>✗</button>

  </div>

  )}

</div>

);

}

```

Task Reminder

☐

meeting (2026-05-08) - Pending ☒ ☒

search (2026-05-21) - Pending ☒ ☒

9. Design a React application that demonstrates the implementation of routing using the react-router-dom library. The application should include the Navigation Menu: Create a navigation bar with links to three distinct pages, Home, About, Contact. Develop separate components for each page (Home, About, and Contact) with appropriate content to differentiate them. Configure routes using react-router-dom to render the corresponding page component based on the selected link. Use BrowserRouter and Route components for routing. Highlight the active link in the navigation menu to indicate the current page.

Step 1: Create a New React Application

```
npx create-react-app my-routing-app          cd my-routing-app
```

```
Step 2: Install react-router-dom          npm install react-router-dom
```

Step 3: update App.js

```
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom';
```

```

const Page = ({ t, c }) => (
  <div style={{ textAlign: 'center' }}>

```

```

    <h2>{t}</h2>

    <p>{c}</p>

  </div>

);

export default function App() {
  return (
    <BrowserRouter>
      <nav style={{ textAlign: 'center', marginBottom: 20 }}>
        <NavLink to="/" style={{ margin: 10 }}>Home</NavLink>
        <NavLink to="/about" style={{ margin: 10 }}>About</NavLink>
        <NavLink to="/contact" style={{ margin: 10 }}>Contact</NavLink>
      </nav>

      <Routes>
        <Route path="/" element={<Page t="Home Page" c="Welcome to Home!" />} />
        <Route path="/about" element={<Page t="About Page" c="About us info" />} />
        <Route path="/contact" element={<Page t="Contact Page" c="Contact us here" />} />
      </Routes>
    </BrowserRouter>
  );
}

```

[Home](#) [About](#) [Contact](#)

Contact Page

Contact us here

10. Design a React application featuring a class-based component that demonstrates the use of lifecycle methods to interact with an external API. The component should fetch and update data dynamically based on user interactions or state changes. Use the `componentDidMount` lifecycle method to fetch data from an API when the component is initially rendered. Display the fetched data in a structured format, such as a table or list. Use the `componentDidUpdate` lifecycle method to

detect changes in the component's state or props. Trigger additional API calls to update the displayed data based on user input or actions (e.g., filtering, searching, or pagination). Implement error handling to manage issues such as failed API requests or empty data responses. Display appropriate error messages to the user when necessary. Allow users to perform actions like filtering, searching, or refreshing the data. Reflect changes in the displayed data based on these interactions.

Step 1: Create a New React Application

```
npx create-react-app data-fetcher          cd data-fetcher
```

step 2:update App.js

```
import React, { Component } from "react";

const API = "https://jsonplaceholder.typicode.com/users";

class App extends Component {
  state = { d: [], f: [], s: "", e: "", l: 0 };

  componentDidMount() { this.get(); }

  componentDidUpdate(_ , p) {
    if (p.s !== this.state.s)
      this.setState({
        f: this.state.d.filter(x =>
          x.name.toLowerCase().includes(this.state.s.toLowerCase())
        )
      });
  }

  get = async () => {
    this.setState({ l: 1, e: "" });
    try {
      let r = await fetch(API);
      let d = await r.json();
      this.setState({ d, f: d, l: 0 });
    }
  }
}
```

```

    } catch {
      this.setState({ e: "Error", l: 0 });
    }
  };

  render() {
    let { f, s, l, e } = this.state;

    return (
      <div>
        <h1>User Data</h1>

        {e} && <p style={{ color: "red" }}>{e}</p>

        <input value={s} onChange={x => this.setState({ s: x.target.value })} />

        {l ? <p>Loading...</p> :
          <table border="1">
            <thead>
              <tr><th>Name</th><th>Email</th><th>City</th></tr>
            </thead>
            <tbody>
              {f.length ? f.map(x =>
                <tr key={x.id}>
                  <td>{x.name}</td>
                  <td>{x.email}</td>
                  <td>{x.address.city}</td>
                </tr>
              ) : <tr><td colspan="3">No results</td></tr>
            </tbody>
          </table>
        }
      </div>
    );
  }
}

```

```

    }

    <button onClick={this.get}>Refresh</button>

  </div>

);
}
}

export default App;

step 3:update index.js

import React from "react";

import ReactDOM from "react-dom/client";

import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(<App />);

```

User Data

Name	Email	City
Leanne Graham	Sincere@april.biz	Gwenborough
Ervin Howell	Shanna@melissa.tv	Wisokyburgh
Clementine Bauch	Nathan@yesenia.net	McKenziehaven
Patricia Lebsack	Julianne.OConner@kory.org	South Elvis
Chelsey Dietrich	Lucio_Hettinger@annie.ca	Roscoeview
Mrs. Dennis Schulist	Karley_Dach@jasper.info	South Christy
Kurtis Weissnat	Telly.Hoeger@billy.biz	Howemouth
Nicholas Runolfsdottir V	Sherwood@rosamond.me	Aliyaview
Glenna Reichert	Chaim_McDermott@dana.io	Bartholomebury
Clementina DuBuque	Rey.Padberg@karina.biz	Lebsackbury
Refresh		